3SA: Semantic Search for Speeches in Audio

Syed Ikram **Venkata Sai Pavan Kumar Kosaraju** **Yifan Li**

{sikram, vkosaraj, yla570}@sfu.ca

CMPT - 825 Natural Language Processing
Final Project Milestone

Abstract

Semantic search is the ability to search for documents by understanding the overall meaning of the query rather than using simple keyword matches. Recent breakthroughs in NLP like Bert, Albert, Roberta, etcetera, paved the way for the development of such powerful semantic search engines. But most of these search algorithms are mainly focused on textual information, i.e., both the document and the query are in natural language. In this project, we aim to develop a semantic search algorithm for arbitrary objects (objects which are not in natural language), specifically for speeches in audio, by leveraging advanced NLP techniques. We introduce 3SA, Semantic Search for Speeches in Audio, which can enable the search for audio files, semantically. We perform our experiments on the Librispeech dataset and further evaluate our search results using basic information retrieval metrics.

1 Approach

The goal of this project is to build a model that takes an audio file as an input and predicts the sentence embeddings of the utterance (speech text) in the file. We can then use this model as an algorithm to represent the audio files in natural language space, thereby allowing us to search for these files semantically.

1.1 Data gathering

Librispeech ASR corpus (Panayotov et al., 2015) suits the best for this task. Librispeech is a large corpus of reading English speech, curated for training and evaluating speech recognition models. It is available to download freely. The corpus contains up to 1000 hours of the recordings from various English speakers reading public domain audiobooks. The dataset is divided into training, development, and testing sets by the curators themselves. The training set is further subdivided into 3 different sets, each of different sizes. Each file contains a random sentence or part of a sentence from the public domain audiobooks, read by one of the 1166 speakers.

1.2 Pre-processing audio files

After gathering data, the next step is to pre-process the audio files. We need to convert our raw audio files into numbers. The idea is to extract representable features for each audio file. In general, audio signals are represented as waveforms (time-distributed amplitudes), but they are not very informative as they only represent the 'loudness' of the recording.

Spectrograms of audio signals have proven to be one of the best feature representations for audio files. As mentioned by Flanagan (1972), trained humans can decode phonemes from spectrogram signals, thus making it easy for any machine learning model to 'learn' to understand these phonemes from spectrograms. Many state-of-the-art acoustic models for ASR tasks like DeepSpeech (Hannun et al., 2014), Kaldi ASR toolkit (Ravenelli et al., 2019), and Google Brain's SpecAugment (Park et al., 2019) use spectrograms as features to perform speech recognition. Hence, we decided to go with spectrogram representations as features for audio files.

The idea behind the spectrogram is to calculate the Fast Fourier Transform (FFT) for each timestep by breaking the audio signal into smaller time frames. By doing so, we get a set of frequencies within each time frame. We should also make sure that there is an overlap between these time frames so that we do not miss out on any frequencies. Having a 20 millisecond time frames with a 50% overlap of each time frame is the most common practice in modern-world ASR tasks. Thus, we

calculate spectrograms for all the audio files that are present in the dataset.

1.3 Get Sentence Embeddings

Next, we need to get sentence embeddings of all the transcripts in the dataset. Sentence embeddings act as truth labels for the 3SA model. One caveat here is that the sentence embeddings should be able to capture the overall semantic meaning of each transcript.

A simple way is to calculate sentence embeddings is to make use of word embeddings like Word2vec, Glove, Fast text, and BERT. Most of these model's vocabulary highly overlaps with Librispeech transcripts vocabulary since they are from English audiobooks. But such sentence embeddings may not always give an accurate semantic representation as per the comment of BERT's author [1].

The other way to go about this is to research those models which perform the best on STS (Semantic Textual Similarity) benchmark dataset – a dataset of human-rated English sentence pairs. Agirre et al., (2017)'s literature review has provided a set of models that perform well on this dataset. One such method is SIF – Smooth Inverse Frequency (Arora et al., 2017), a technique to get sentence embedding by calculating weighted averaged of respective word embeddings. SIF has showcased some great results on the STS dataset.

1.4 Build and train the model

After having our dataset prepared, we then build our model which takes spectrograms as input and predicts the respective sentence embeddings. Most audio files in Librispeech are of different results. This results in generating spectrograms of different lengths for different audio files. So, we need to utilize RNNs in our model so that they can take any length inputs. Then we use the hidden layer as the encoded version of the audio file while ignoring the outputs. We then pass the hidden layer's output through some dense layers and predict the sentence embedding in the final layer. A simple Cosine similarity can act as a loss function for our model. We perform forward-pass our training data and backpropagate the error to learn the model weights.

1.5 Evaluation of the model

To evaluate our model, we use Precision @ K metric. As mentioned in our project proposal, evaluation is the trickiest part of this project.

Unlike the STS benchmark dataset, we do not have any human-rated audio-sentence pairs for Librispeech. Since many factors can affect the evaluation, we develop a small evaluation set and stick with the primary information retrieval metric, Precision @ K, to limit the scope of our project.

Here we consider the test set of the Librispeech corpus. We randomly pick at least 50 (as time permits) audio files X and their transcripts Y, respectively. Then, for each transcript in Y, we paraphrase the transcript and get a paraphrased set Y containing paraphrased sentences. Then we generate sentence embeddings for this new set Y to get $Y^{\#}$.

Next, we feed X to our model and predict sentence embeddings Y^* and index the audio files using it. Now for each paraphrased transcript embedding $y^{(i)}$ in $Y^{\#}$, we get the K - closest neighbors in the set Y^* and their respective audio files (since they are now indexed with Y^*). Now we say that it is true positive if the audio file which is responsible for generating $y^{(i)}$ is among those neighbors. Thus, we calculate Precision @ K, for K = 1,3,5, and 10 for our model.

2 Experiments & Progress

We already accomplished the tasks mentioned in sections 1.1 and 1.2. We also had some premature progress with sections 1.3 and 1.4. The progress is available in the notebook at:

https://csil-git1.cs.surrey.sfu.ca/sikram/nlpclass-1201-g-semantic_seekers/-/blob/master/project/milestone.ipynb

(Note: The notebook currently is not appropriately documented but will be updated it as we progress)

2.1 Data gathering with torchaudio

Torch audio is the latest addition to the PyTorch framework for audio related tasks. The library also provides pre-built datasets related to ASR tasks, one of them being Librispeech. We use torch audio to get all our data. The library provides training, development, and test sets. We downloaded and gathered all three datasets for our experiments. For a given index, we can fetch the waveform of the audio signal in tensors format along with the respective transcript with few miscellaneous information like sample rate and utterance ID.

[1] BERT's Author Jacob Devlin's comment here GitHub Issues Link

2.2 Pre-processing with torchaudio

Torch audio also provides preprocessing tools to preprocess our audio files. With Spectrogram class in the torch library, we can get the spectrograms for a given waveform. We intend to use this library by incorporating it into our model's architecture.

2.3 Sentence Embeddings

We utilized BERT embeddings and calculated sentence embeddings for few transcripts in our dataset. Upon performing a sanity check, the results seemed promising.

We also tried the Fast Sentence Embeddings (fse) library, to get SIF embeddings for our audio transcripts. We used Fast Text's word embeddings to calculate SIF based sentence embeddings. Even this approach showed some good results.

2.4 Basic Model architecture

We also built a preliminary model for our project. The model consists of a GRU layer and few dense layers. We will improve our model's structure as we move forward in our project.

3 Future Work & Plans

Since our SIF-based sentence embeddings are shorter in length, we are planning to train our model with these embeddings so that our model can converge faster. Later, we use BERT based embeddings to train our model to check for improvements.

Before we train our model, we intend to make some final tweaks by adding activation and batch normalization layers accordingly to our model and finalize the architecture.

Parallelly, we devise a way to generate paraphrased transcripts either manually or by leveraging advanced NLP-based paraphrase tools. Once we have paraphrased sentences, we can easily generate respective sentence embeddings and evaluate our model as described in section 1.5.

Acknowledgments

We would like to sincerely thank Frome et al., (2013), the authors of the paper *DeVISE: A Deep Visual-Semantic Embedding Model*, which inspired us to carry out this research.

We would like to thank Olive Borchers for fse (Fast Sentence Embeddings) library. GitHub repo can be found here.

We would also like to thank Chris McCormick and Nick Ryan for their article *BERT Word Embeddings Tutorial* which helped us calculate BERT-based sentence embeddings. Link here.

We would like to thank Kartik Chaudhary for the article *Understanding Audio data, Fourier Transform, FFT and Spectrogram features for a Speech Recognition System.* Link here.

Finally, we would like to thank Anoop Sarkar and Angel Xuan Chang for the CMPT-825 NLP assignments *Robust Phrasal Chunking* and *Attention* which helped us understand the working of RNNs and allowed us to build the model architecture for our project.

References

Agirre E, Cer D, Diab M, Lopez-Gazpio I, Specia L. (2017). *Semeval-2017 Task 1: Semantic Textual Similarity Multilingual and Crosslingual Focused Evaluation*. Proceedings of SemEval 2017.

Arora S, Liang Y, Ma T (2017). *A Simple but Tough-to-Beat Baseline for Sentence Embeddings*. Int. Conf. Learn. Represent. (Toulon, France).

Flanagan J. (1972). *Speech Analysis, Synthesis and Perception*. Springer- Verlag, New York.

Frome A, Corrado G, Shlens J, Bengio S, Dean J, Ranzato M, Mikolov T. (2013). *DeViSE: A Deep Visual-Semantic Embedding Model*. NIPS 2013.

Hannun A, Case C, Casper J, Catanzaro B, Diamos G, Elsen E, Prenger R, Satheesh S, Sengupta S, Coates A, Ng A. *Deep speech: Scaling up end-to-end speech recognition*. .arXiv preprint arXiv:1412.5567, 2014.

Panayotov V, Chen G, Povey D, Khudanpur S. (2015). *LibriSpeech: an ASR corpus based on public domain audio books*. ICASSP 2015.

Park D, Chan W, Zhang Y, Chiu C, Zoph B, Cubuk E, Le Q. (2019). *SpecAugment: A Simple Data Augmentation Method for Automatic Speech Recognition.* Proc. Interspeech 2019, 2613-2617.

Povey D, Ghoshal A, Boulianne G, Burget L, Glembek O, Goel N, Hannemann M, Motlicek P, Qian Y, Schwarz P, Silovsky J, Stemmer G, Vesely K. 2011. *The Kaldi Speech Recognition Toolkit*. IEEE 2011 workshop on Automatic Speech Recognition and Understanding. (Hawaii, US). record such decisions to precisely characterize state-of-the-art methods.